

Multiatendimento Mobile

Datacrazy

GABRIEL F. HERMENEGILDO

SUMÁRIO

1. Introdução e escopo.....	3
2. Estrutura de navegação.....	3
3. Por que offline first.....	4
4. Onde vive o histórico e por que não em memória.....	5
4.1. Validação do WatermelonDB.....	5
5. Responsabilidade de cada camada.....	6
6. Identificação única de mensagem.....	7
7. Fluxo de envio e recebimento.....	7
8. Ordenação de mensagens.....	8
9. Sincronização ao abrir o app.....	8
10. Canal de tempo real e notificações.....	9
11. Envio de foto e arquivo.....	10
12. Mensagens rápidas.....	11
13. Controle de automação na conversa.....	11
14. Riscos e dependências de backend.....	12
15. Resumo das decisões.....	13
16. Estimativa e estratégia de entrega.....	14
17. Conclusão.....	15

1. Introdução e escopo

Este documento detalha a proposta técnica para tornar o multiatendimento mais fluido no aplicativo mobile. Para mim, essa é a funcionalidade que mais justifica a existência do Datacrazy como aplicativo e não apenas como um site responsivo, porque o atendente precisa acompanhar e responder conversas durante o dia mesmo quando a conexão do celular não é estável.

A proposta parte de duas decisões definidas nos documentos anteriores. A primeira é manter React Native como base, então a arquitetura abaixo é pensada para ser implementada no projeto atual sem uma migração completa para Kotlin e Swift. A segunda é manter aberta a decisão entre a base React Native CLI atual e a adoção de Expo Modules sobre essa mesma base. Por isso, quando uma biblioteca muda entre esses dois cenários, isso aparece explicitamente.

Existe também uma limitação importante de teste. Não foi possível testar envio e recebimento real de mensagens, porque isso depende de conexão ativa com WhatsApp, Instagram ou outro canal e de uma conta business que não estava disponível no ambiente. A análise cobre navegação, filtros e estrutura da tela. As decisões de arquitetura para o comportamento do chat precisam ser validadas contra o backend real antes da implementação.

2. Estrutura de navegação

A tela de lista de conversas já existente no app está bem resolvida e eu a manteria como base. Ela possui filtros rápidos por status, filtro avançado e seleção em massa. Existe, porém, um ponto que vale entrar na estabilização: há relato público de que o contador das abas de status pode mostrar um número diferente do real, como cinco conversas não iniciadas quando existem duas. Isso não apareceu durante o teste porque a conta utilizada estava zerada nas abas.

Uma causa comum desse tipo de divergência é a contagem vir de um endpoint separado da listagem, sem invalidação quando uma ação muda o status da conversa. Eu verificaria isso junto da correção geral de sincronização de dados, em vez de criar uma correção isolada apenas na interface.

A mudança estrutural principal está na conversa individual. Em vez de reproduzir as três áreas do web ao mesmo tempo, lista de conversas, conversa e dados do contato, o mobile deve ocupar a tela inteira com a conversa. O cabeçalho mostra foto e nome do contato e pode ser tocado para abrir a tela de detalhe do lead que já existe no aplicativo.

Reaproveitar a tela de detalhe existente evita duplicar a exibição das informações do contato. Quando o lead tiver mais de um negócio vinculado, um badge discreto pode aparecer junto ao nome, por exemplo "4 negócios", dando contexto ao atendente sem exigir uma navegação adicional.

Mensagens de automação devem ter uma identificação visual diferente, por exemplo uma variação de balão ou um selo. Isso mantém a mesma ideia visual do CRM web e é importante porque o atendente precisa saber quando está diante de uma mensagem gerada automaticamente e quando uma intervenção manual é necessária.

3. Por que offline first

Chat é um dos casos de uso em que offline first mais se justifica. O celular pode perder e recuperar conexão em elevador, túnel ou área com sinal fraco, e o usuário espera continuar lendo o histórico e digitando mesmo sem rede. Além disso, esperar a resposta do servidor antes de mostrar uma mensagem que o próprio atendente acabou de enviar cria uma sensação de lentidão que não precisa existir.

A proposta é inverter a prioridade: o app lê e escreve no banco local primeiro e sincroniza com o servidor depois. O banco local passa a representar o estado que a interface mostra, enquanto a rede funciona como mecanismo de sincronização e comunicação com o servidor.

Vale deixar claro o custo dessa decisão. Esta é a parte mais cara e de maior risco técnico de toda a proposta, estimada em 8 a 10 semanas de esforço bruto. Ela concentra decisões de arquitetura que não se dividem perfeitamente entre várias pessoas. Se for necessário comprimir o prazo, é possível entregar primeiro leitura e escrita locais e sincronização ao abrir o app, deixando anexos e uma sincronização incremental mais refinada para uma segunda etapa.

O que eu não faria é construir o chat sobre estado em memória e tentar adicionar persistência depois. Nesse cenário, boa parte da camada de dados teria de ser refeita justamente quando o produto já estivesse dependendo dela.

4. Onde vive o histórico e por que não em memória

Conversas e mensagens devem ficar em um banco local estruturado em SQLite. O histórico pode chegar a milhares de mensagens e não faz sentido manter esse volume em memória por meio de Zustand ou outra store global. Além do consumo de RAM, isso aumenta o risco de perder estado quando o aplicativo é fechado ou sofre um crash.

Minha recomendação é usar WatermelonDB sobre SQLite. A biblioteca resolve três partes que seriam caras de construir do zero neste projeto: observabilidade reativa, carregamento preguiçoso e um protocolo de sincronização baseado em pull e push com uma marca de última sincronização.

A observabilidade reativa é especialmente útil aqui. Quando uma mensagem chega ou uma mensagem pendente muda de estado, a interface pode reagir à alteração persistida no banco, em vez de manter uma segunda representação do mesmo dado apenas para atualizar a tela.

Na base React Native CLI atual, o adaptador considerado é `@op-engineering/op-sqlite`. No cenário com Expo Modules, a alternativa é `expo-sqlite`. O `react-native-quick-sqlite`, que aparece em materiais mais antigos, foi depreciado pelos próprios mantenedores, que apontam para `react-native-nitro-sqlite`.

4.1. Validação do WatermelonDB

A recomendação do WatermelonDB é condicionada a uma prova de conceito. A biblioteca está há cerca de um ano sem publicação nova, com último commit citado no levantamento em agosto de 2025. Isso, isoladamente, não significa que esteja abandonada, porque bibliotecas maduras podem ter ciclos de manutenção espaçados.

O ponto mais importante é a compatibilidade com a versão exata do aplicativo. Existem relatos de atrito com a Nova Arquitetura em versões recentes do React Native, e o Datacrazy está em React Native 0.82. Como o WatermelonDB ficaria no centro da

arquitetura, eu não começaria seis semanas de implementação sem validar essa combinação.

A prova de conceito deve durar de dois a três dias e ter um escopo pequeno: criar o esquema, gravar e ler alguns milhares de registros, confirmar que a observabilidade reativa funciona com Fabric e executar um ciclo de sincronização simulado. Se passar, o WatermelonDB segue como escolha recomendada.

Se falhar, o plano B é acesso direto ao SQLite com uma camada de repositório própria, emitindo mudanças após cada escrita e mantendo hooks assinados nessa emissão. É mais código do time, mas fica sob controle da equipe. O custo adicional estimado é da ordem de uma a duas semanas.

Independentemente do resultado da prova de conceito, o acesso ao banco deve ficar isolado atrás de um repositório próprio. Isso torna a decisão reversível e evita que uma eventual troca de mecanismo se espalhe pelo restante do aplicativo.

5. Responsabilidade de cada camada

O banco local deve ser a única fonte de verdade do histórico de chat. A interface renderiza a partir dele, nunca diretamente de eventos de rede ou de outra camada de estado. Essa regra evita que o mesmo dado exista em paralelo no banco, no cache de rede e no estado global, criando inconsistências difíceis de reproduzir.

A partir dessa regra, cada ferramenta fica com uma responsabilidade bem definida.

- Banco local: mensagens, conversas, fila de envio pendente, catálogo de mensagens rápidas e estado de automação por conversa. Tudo que precisa sobreviver ao fechamento do app, a um crash ou a um período prolongado offline.
- Zustand: apenas estado de sessão e estado efêmero, como qual conversa está aberta e o status da conexão, por exemplo conectando, sincronizando ou conectado.
- TanStack Query: requisições e mutações de API que não representam o histórico do chat em si, como dados complementares do lead e outras telas do aplicativo.

A fila de mensagens pendentes deve estar no banco local e não apenas em uma store persistida. Dessa forma, a fila sobrevive de maneira transacional se o aplicativo fechar abruptamente no meio de um envio.

6. Identificação única de mensagem

Cada mensagem criada no dispositivo recebe um identificador único no momento em que é criada. Esse identificador é enviado ao servidor junto com a mensagem. Se o app perder a resposta de uma requisição e precisar tentar novamente, o servidor consegue reconhecer que se trata da mesma mensagem e não cria uma duplicata.

Esse mesmo princípio de idempotência por identificador de tentativa deve ser usado nas mutações críticas relacionadas ao atendimento. Isso inclui envio de mensagem, envio de anexo e interrupção de automação. O mesmo padrão já foi proposto para movimentação de negócio entre etapas no kanban.

7. Fluxo de envio e recebimento

Ao enviar uma mensagem, ela é gravada imediatamente no banco local com status pendente. A interface mostra o balão na hora, sem esperar confirmação da rede. O app tenta o envio pelo canal disponível e, quando o servidor confirma, o estado local avança para enviado e depois entregue, seguindo o modelo que o usuário já conhece de outros aplicativos de mensagem.

Ao receber uma mensagem, o evento também é gravado primeiro no banco local. A interface observa essa alteração e se atualiza a partir do estado persistido. Assim, mesmo que o aplicativo seja fechado logo depois, a tela continuará partindo da mesma informação que já foi salva.

A análise do aplicativo tornou obrigatório que falhas sejam visíveis. Foi observado que, com plano expirado, algumas ações como editar nota, perfil e endereço simplesmente não salvam sem apresentar uma mensagem adequada. No chat isso seria muito mais grave, porque o atendente poderia acreditar que respondeu um cliente quando a mensagem nunca chegou ao servidor.

Toda mensagem que esgotar as tentativas de envio deve apresentar estado de falha claro no próprio balão, com ação de tentar novamente. O motivo precisa ser legível para o usuário, e não um código bruto retornado pela API.

8. Ordenação de mensagens

Timestamp sozinho não é suficiente para garantir a ordem correta quando duas mensagens chegam quase no mesmo instante ou por canais diferentes. A solução mais confiável é o servidor fornecer um número de sequência incremental por conversa, e o aplicativo usar essa sequência como critério definitivo de ordenação. O timestamp continua sendo usado para exibição de data e hora.

Esse cuidado é ainda mais importante neste projeto porque a análise documentou um problema no relógio e no fuso horário, com atividades criadas às 22h ou 23h aparecendo no dia seguinte. Enquanto essa inconsistência não for corrigida, ordenar mensagens pelo horário poderia produzir uma ordem errada sem deixar evidente a causa.

A sequência do servidor protege a ordem da conversa, mas não corrige a exibição de data e hora. A correção do fuso precisa continuar na fase de estabilização, antes da implementação completa do multiatendimento.

9. Sincronização ao abrir o app

Ao abrir o app, a tela deve mostrar imediatamente o que já está salvo localmente. Em paralelo, o aplicativo conecta ao servidor e busca somente o que mudou desde a última sincronização conhecida. Para isso, a sequência ou marca de sincronização anterior funciona como referência, evitando baixar todo o histórico novamente.

O protocolo de sincronização do WatermelonDB já trabalha com pull do que mudou desde a última marca e push das mudanças locais pendentes. A implementação, portanto, precisa adaptar esse mecanismo ao contrato que a API do Datacrazy realmente oferece.

O `@react-native-community/netinfo` monitora o estado da rede e pode disparar a mesma rotina quando a conexão voltar. A biblioteca é a mesma tanto na base bare quanto no cenário com Expo Modules.

Existe uma dependência importante de backend. A sincronização incremental pressupõe que a API consiga responder algo equivalente a "o que mudou desde X". Se hoje o backend só devolver listas completas com paginação, esse contrato precisa ser negociado antes da implementação. Esse é o principal risco de bloqueio externo desta fase.

10. Canal de tempo real e notificações

Enquanto o aplicativo está aberto, a conexão em tempo real mantém as mensagens chegando sem que o app precise consultar repetidamente se existe algo novo. Quando o aplicativo está fechado ou em segundo plano, essa conexão não é uma garantia. Nesse momento, a notificação push assume o papel de avisar o usuário e levá-lo de volta à conversa correta.

Hoje, nas Preferências do app, notificações aparecem como recurso ainda não ativo. Considerando que responder rapidamente é uma das principais justificativas para ter o atendimento no celular, eu colocaria push entre as primeiras prioridades desta fase.

Para recebimento, a proposta é `@react-native-firebase/messaging` integrado ao Firebase Cloud Messaging nos dois cenários de infraestrutura. Para exibição, canais e comportamento, a opção na base bare é `react-native-notify-kit`; com Expo Modules adotado, `expo-notifications` passa a ser a alternativa mais simples.

O `react-native-notify-kit` é o fork comunitário do `Notifee`, que foi arquivado pela Invertase em abril de 2026 e apontou alternativas em seu próprio projeto. O `expo-notifications` possui manutenção institucional, mas não cobre `foreground service` no Android. Isso só passa a ser decisivo se o roadmap exigir uma notificação persistente de atendimento em curso.

Ao tocar na notificação, o app deve abrir a conversa correta usando o linking do React Navigation. O payload precisa carregar o identificador da conversa.

A permissão de notificação precisa ser tratada explicitamente no Android 13 ou superior e no iOS. O token do dispositivo também precisa ser registrado e atualizado no backend por usuário e invalidado no logout. Caso contrário, um atendente que sair da conta pode continuar recebendo notificações da conta anterior.

Esse ponto se conecta a um problema já documentado no aplicativo: depois da troca de senha, existe um cenário em que a sessão fica inconsistente e o logout não acontece como deveria. Enquanto a invalidação da sessão não estiver correta, a invalidação do token de push também precisa ser tratada com cuidado.

Não considero necessário depender de execução em background para garantir o envio. No Android, o `WorkManager` oferece mecanismos de execução agendada, enquanto o

equivalente no iOS é oportunista e o sistema decide quando pode executar. Essa diferença existe independentemente de React Native. Por isso, a arquitetura usa a fila local e retoma o envio quando o app volta ao primeiro plano ou quando a rede retorna, enquanto o push cobre o período em que o app está fechado.

11. Envio de foto e arquivo

Há relato público de que não era possível enviar arquivo ou foto pelo atendimento. A empresa informou que a limitação de acesso à galeria havia sido corrigida na versão 1.0.79, de julho de 2026, então o problema específico de permissão provavelmente está resolvido. O motivo para manter anexos nesta proposta é outro: arquivo quebra algumas premissas da fila de envio e exige decisões de arquitetura próprias.

Na base bare, a seleção de mídia pode usar `react-native-image-picker` e `@react-native-documents/picker`. No cenário com Expo Modules, as alternativas são `expo-image-picker` e `expo-document-picker`. A captura direta pela câmera pode usar a mesma biblioteca de imagem.

O arquivo não deve ser colocado dentro do registro do banco local. O outbox guarda o caminho local do arquivo. Esse arquivo precisa ser copiado para um diretório próprio da aplicação, porque o caminho original da galeria pode deixar de existir antes que o envio termine.

Uploads grandes também precisam de tratamento diferente de uma mensagem de texto. O envio deve suportar retomada ou, no mínimo, uma nova tentativa completa usando o mesmo identificador de idempotência, evitando anexo duplicado se a conexão cair no meio.

A interface precisa mostrar progresso. Os estados pendente, enviado e entregue usados para texto não são suficientes para um arquivo. Um anexo parado em 40 por cento comunica uma situação diferente de um arquivo que ainda está aguardando para começar.

Também é necessário ter uma política de consumo de dados. Em uma rede móvel, enviar automaticamente um arquivo grande pode consumir a franquia do usuário. Uma opção é segurar anexos acima de determinado tamanho até que o dispositivo esteja no Wi-Fi, permitindo uma ação explícita para forçar o envio.

Os arquivos copiados para o diretório da aplicação precisam ser removidos depois da confirmação do envio, evitando acúmulo de armazenamento. Também vale validar tamanho e formatos aceitos por cada canal externo antes do envio, já que WhatsApp, Instagram e Facebook possuem regras próprias.

12. Mensagens rápidas

Há relato público (avaliação da loja) de que nem todas as mensagens rápidas criadas aparecem no aplicativo. Esse recurso não estava mapeado na análise inicial, mas ele é particularmente valioso no mobile porque digitar respostas longas em um teclado de celular é um dos maiores atritos do atendimento.

Dentro da conversa, eu colocaria um acesso rápido ao catálogo por bottom sheet, com busca por texto. A mensagem selecionada é inserida no campo de digitação em vez de ser enviada imediatamente, permitindo que o atendente ajuste o conteúdo antes de mandar.

Se existirem variáveis de substituição, como o nome do lead, a substituição acontece no momento da inserção e o atendente vê o texto final antes do envio.

O catálogo é um dado pequeno e de baixa volatilidade, então pode viver no banco local junto do restante do atendimento e ficar disponível offline. A busca também acontece localmente, sem chamada de rede a cada tecla. Criar e organizar o catálogo continua sendo responsabilidade da web, porque são tarefas de configuração e não de operação diária.

Sobre o relato de mensagens faltando, as causas prováveis incluem paginação não tratada no cliente, filtro divergente entre o endpoint do app e o da web ou restrição por departamento aplicada de forma diferente. Esse caso merece ser investigado junto do problema já documentado de produtos com mais negócios, que aparecia vazio no app e preenchido no web. Os dois podem indicar uma divergência mais ampla entre endpoints.

13. Controle de automação na conversa

Há relato público de ausência de acesso à automação no aplicativo. O app já possui a ação "executar automação" dentro da seleção em massa de conversas, mas ela está

disponível apenas em lote e escondida atrás desse fluxo. Para uma conversa individual, falta acesso exatamente no ponto em que o atendente precisa agir.

Eu manteria essa primeira versão deliberadamente enxuta, com três ações.

1. Disparar uma automação existente na conversa aberta, escolhendo a automação em uma lista pelo mesmo padrão de bottom sheet usado em outros fluxos.
2. Mostrar que uma automação está em andamento, aproveitando a sinalização visual de mensagens automatizadas.
3. Interromper a automação em andamento. Esta é a ação mais importante, porque o momento em que o atendente precisa assumir manualmente é justamente o momento em que ele pode não estar diante do computador.

Criar automação, editar condições, alterar gatilhos e configurar ramificações continuam na web. Isso não é uma limitação técnica. São fluxos com muitos passos, alto impacto e baixa frequência, exatamente o tipo de operação que fica pior em uma tela pequena.

A implementação consome os endpoints de automação já existentes e mantém o estado da automação por conversa no banco local. A interrupção usa mutação otimista com rollback e identificador único por tentativa, seguindo o mesmo padrão usado para outras operações críticas.

14. Riscos e dependências de backend

O maior risco técnico está na sincronização, não na tela de conversa. Um chat offline first exige que o cliente consiga reconciliar mudanças locais e remotas sem duplicar mensagens, perder alterações ou apresentar um estado diferente do servidor.

O maior risco externo é o contrato de sincronização incremental. Se a API não tiver uma forma de recuperar as alterações desde uma sequência conhecida, o backend precisará fornecer essa capacidade ou uma estratégia equivalente antes que a arquitetura seja considerada concluída.

A segunda área de risco é a compatibilidade do banco local com React Native 0.82 e a Nova Arquitetura. Por isso a prova de conceito do WatermelonDB vem antes da implementação da camada inteira.

O terceiro ponto é a manutenção das dependências de dispositivo, principalmente notificações, seleção de mídia e documentos. A solução é manter essas dependências atrás de interfaces próprias, evitando que componentes da aplicação inteira dependam diretamente de uma biblioteca específica.

15. Resumo das decisões

Decisão	Escolha	Motivo principal
Fonte de verdade do histórico	Banco local SQLite	Evita duplicação de estado e permite abrir a conversa sem rede.
Camada de dados	WatermelonDB, condicionado à POC	Reatividade, carregamento preguiçoso e sincronização pull/push; validar em RN 0.82.
Adaptador SQLite	@op-engineering/op-sqlite ou expo-sqlite	Alternativas conforme a infraestrutura escolhida.
Plano B da camada de dados	SQLite direto com repositório próprio	Se a POC do WatermelonDB falhar; custo adicional estimado de 1 a 2 semanas.
Isolamento	Repositório próprio sobre a camada de dados	Torna a escolha reversível.
Fila de envio	Outbox no banco	Sobrevive a crash e fechamento abrupto.
Deduplicação	Identificador local por tentativa	Retentativas não criam mensagens duplicadas.
Ordenação	Número de sequência do servidor	Timestamp não é confiável para ordenar.
Sincronização	Incremental por sequência, ao abrir e ao recuperar conexão	Evita baixar o histórico inteiro novamente.

Tempo real e push	Tempo real com app aberto, push quando fechado	Não depende de background oportunista no iOS.
Anexos	Caminho no outbox, arquivo em diretório próprio e progresso visível	Upload tem falhas e estados diferentes de texto.
Mensagens rápidas	Catálogo local com busca e inserção no campo	Funciona offline e reduz digitação no celular.
Automação	Disparar, visualizar estado e interromper	Foco nas ações de operação que realmente precisam do mobile.

16. Estimativa e estratégia de entrega

A fase de multiatendimento está estimada em 8 a 10 semanas de esforço bruto de desenvolvimento. É a fase mais pesada da proposta porque envolve persistência local, sincronização, outbox, realtime, push, anexos e tratamento de estados que precisam continuar corretos mesmo com rede instável.

Eu dividiria a execução em etapas, com a prova de conceito do banco acontecendo antes de comprometer a arquitetura principal.

1. POC do WatermelonDB, com duração de dois a três dias, validando React Native 0.82, Nova Arquitetura, Fabric, alguns milhares de registros e sincronização simulada.
2. Modelagem local e camada de repositório, definindo conversas, mensagens, outbox, mensagens rápidas e estado de automação.
3. Fluxo de leitura e escrita local, com atualização otimista e estados pendente, enviado, entregue e falha.
4. Sincronização incremental e reconciliação com o backend, incluindo sequência por conversa e idempotência.
5. Tempo real e push, incluindo linking para a conversa correta, permissões e ciclo de vida do token.

6. Anexos, mensagens rápidas e controle de automação.
7. Testes em aparelhos reais, principalmente cenários de perda de conexão, fechamento abrupto, retentativa, duplicidade e histórico grande.

Se for necessário entregar uma primeira versão antes das 8 a 10 semanas completas, eu priorizaria a conversa com leitura e escrita locais, sincronização ao abrir o app, fila de texto e push. Anexos e sincronização incremental mais sofisticada poderiam seguir como uma segunda etapa. O que eu evitaria é lançar um chat que pareça offline, mas ainda dependa de estado em memória para funcionar.

17. Conclusão

A proposta é transformar o multiatendimento em uma experiência realmente adequada ao celular, e não apenas reduzir a interface web. A conversa ocupa a tela inteira, o detalhe do lead continua sendo reaproveitado, e as ações mais importantes ficam acessíveis sem exigir que o atendente navegue por estruturas pensadas para desktop.

Tecnicamente, o ponto central é fazer o banco local ser a fonte de verdade do histórico. WatermelonDB sobre SQLite é a escolha recomendada, mas condicionada a uma prova de conceito com React Native 0.82 e Nova Arquitetura. Zustand fica restrito ao estado efêmero, TanStack Query às operações de API que não representam o histórico, e o outbox garante que mensagens pendentes sobrevivam ao fechamento do aplicativo.

A sincronização incremental por sequência, a idempotência e a ordenação pelo número de sequência do servidor são importantes para evitar os problemas mais difíceis desse tipo de aplicação. Tempo real mantém a conversa atualizada enquanto o app está aberto, enquanto o push cobre o período em que ele está fechado.

Por fim, eu trataria anexos, mensagens rápidas e controle de automação como extensões da mesma arquitetura, e não como recursos independentes. Isso permite que o aplicativo continue funcionando de forma previsível quando a rede falhar, quando uma requisição precisar ser repetida ou quando o usuário fechar o app no meio de uma operação.

O resultado esperado é um atendimento que não dependa de conectividade perfeita para parecer rápido e confiável. A rede passa a ser uma camada de sincronização, e não uma condição para a interface funcionar.